| | | |
|---|---|---|
| Marwadi University | **Marwadi University**<br>**Faculty of Technology**<br>**Department of Information and Communication Technology** | |
| **Subject: Artificial Intelligence (01CT0616)** | **Aim:** To study N-gram model | |
| **Experiment No: 7** | **Date:** | **Enrolment No: 92301733012** |

**Aim:** To study N-gram model

**IDE:** Google Colab **Theory:**

Given a sequence of N-1 words, an N-gram model predicts the most probable word that might follow this sequence. It's a probabilistic model that's trained on a corpus of text. Such a model is useful in many NLP applications including speech recognition, machine translation and predictive text input.

An N-gram model is built by counting how often word sequences occur in corpus text and then estimating the probabilities. Since a simple N-gram model has limitations, improvements are often made via smoothing, interpolation and backoff. An N-gram model is one type of a Language Model (LM), which is about finding the probability distribution over word sequences.

Consider two sentences: "There was heavy rain" vs. "There was heavy flood". From experience, we know that the former sentence sounds better. An N-gram model will tell us that "heavy rain" occurs much more often than "heavy flood" in the training corpus. Thus, the first sentence is more probable and will be selected by the model.

A model that simply relies on how often a word occurs without looking at previous words is called unigram. If a model considers only the previous word to predict the current word, then it's called bigram. If two previous words are considered, then it's a trigram model.

An n-gram model for the above example would calculate the following probability:

P('There was heavy rain') = P('There', 'was', 'heavy', 'rain') = P('There')P('was'|'There')P('heavy'|'There was')P('rain'|'There was heavy')

Since it's impractical to calculate these conditional probabilities, using Markov assumption, we approximate this to a bigram model: P('There was heavy rain') ~ P('There')P('was'|'There')P('heavy'|'was')P('rain'|'heavy') In speech recognition, input may be noisy and this can lead to wrong speech-to-text conversions.

N-gram models can correct this based on their knowledge of the probabilities. Likewise, N-gram models are used in machine translation to produce more natural sentences in the target language.

When correcting for spelling errors, sometimes dictionary lookups will not help. For example, in the phrase "in about fifteen mineuts" the word 'minuets' is a valid dictionary word but it's incorrect in this context. N-gram models can correct such errors.

N-gram models are usually at word level. It's also been used at character level to do stemming, that is, separate the root word from the suffix. By looking at N-gram statistics, we could also classify languages or differentiate

| | | |
|---|---|---|
| Marwadi University | **Marwadi University**<br>**Faculty of Technology**<br>**Department of Information and Communication Technology** | |
| **Subject: Artificial Intelligence (01CT0616)** | **Aim:** To study N-gram model | |
| **Experiment No: 7** | **Date:** | **Enrolment No: 92301733012** |

between US and UK spellings. For example, 'sz' is common in Czech; 'gb' and 'kp' are common in Igbo. In general, many NLP applications benefit from N-gram models including part-of-speech tagging, natural language generation, word similarity, sentiment extraction and predictive text input.

To preprocess your text simply means to bring your text into a form that is predictable and analyzable for your task. A task here is a combination of approach and domain. Machine Learning needs data in the numeric form. We basically used encoding technique (Bag Of Word, Bi-gram, n-gram, TF-IDF, Word2Vec) to encode text into numeric vector. But before encoding we first need to clean the text data and this process to prepare (or clean) text data before encoding is called text preprocessing, this is the very first step to solve the NLP problems.

**Program (Code):**

To be attached with

1. N-Gram Model Code for generating dynamic N-Grams

```
corpus = """
I like machine learning
I like deep learning
I enjoy machine learning
I enjoy data science
Machine learning is interesting
Deep learning is powerful
"""
import nltk
from nltk.util import ngrams
from collections import defaultdict, Counter
import random
nltk.download('punkt')
nltk.download('punkt')
nltk.download('punkt_tab')
tokens=nltk.word_tokenize(corpus.lower())
#build ngram model
n=2
bigrams=list(ngrams(tokens, n))
print(bigrams)
#create the probability model
model=defaultdict(Counter)
for w1, w2 in bigrams:
  model[w1][w2]+=1
model

#function to predict the next word

def predict_next_word(word):
```

| | | |
|---|---|---|
| Marwadi University | **Marwadi University**<br>**Faculty of Technology**<br>**Department of Information and Communication Technology** | |
| **Subject: Artificial Intelligence (01CT0616)** | **Aim:** To study N-gram model | |
| **Experiment No: 7** | **Date:** | **Enrolment No: 92301733012** |

```
  word=word.lower()
  if word in model:
    next_words=model[word]
    prediction=max(next_words, key=next_words.get)
    return prediction

  else:
      return "No prediction is done"


predict_next_word("I")
```

**Results:**

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
True
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
```

```
[('i', 'like'), ('like', 'machine'), ('machine', 'learning'), ('learning', 'i'), ('i', 'like'), ('like', 'deep'), ('deep', 'learning'), ('learning', 'i'), (
```

```
defaultdict(collections.Counter,
            {'i': Counter({'like': 2, 'enjoy': 2}),
             'like': Counter({'machine': 1, 'deep': 1}),
             'machine': Counter({'learning': 3}),
             'learning': Counter({'i': 3, 'is': 2}),
             'deep': Counter({'learning': 2}),
             'enjoy': Counter({'machine': 1, 'data': 1}),
             'data': Counter({'science': 1}),
             'science': Counter({'machine': 1}),
             'is': Counter({'interesting': 1, 'powerful': 1}),
             'interesting': Counter({'deep': 1})})
```

| Marwadi University | **Marwadi University**<br>**Faculty of Technology**<br>**Department of Information and Communication Technology** | |
|---|---|---|
| **Subject: Artificial Intelligence (01CT0616)** | **Aim:** To study N-gram model | |
| **Experiment No: 7** | **Date:** | **Enrolment No: 92301733012** |

```
...   'like'
```

**Observation:**

The N-gram model was implemented to understand how word sequences are predicted based on probability. It was observed that the model predicts the next word using previous word(s) context. In unigram, prediction depends only on word frequency, while in bigram and trigram, previous words improve accuracy. The probability of a sentence is calculated as the product of conditional probabilities of words. The model works on the Markov assumption, where only limited previous words are considered. It was also observed that higher order N-grams (like trigram) give better and more meaningful predictions compared to unigram.

**Post Lab Exercise:**

Apply the probability distribution concept to predict the next word by taking sample document and seed phrases. Attach the code and screenshot. Write your observation here.

Code:

```
text = """I am passionate about exploring new technologies and constantly improving my skills in the field of computer science.
 I enjoy solving problems and understanding how systems work behind the scenes.
  Learning new concepts and applying them in practical projects motivates me to grow further.
   I have a strong interest in developing innovative solutions and staying updated with the latest advancements in technology.
"""
import nltk
nltk.download('punkt')

from nltk.tokenize import word_tokenize
from collections import defaultdict

tokens = word_tokenize(text.lower())

# Create Bigram Model
bigrams = list(nltk.bigrams(tokens))
# Create frequency dictionary
freq_dict = defaultdict(lambda: defaultdict(int))
```

| Marwadi University | **Marwadi University**<br>**Faculty of Technology**<br>**Department of Information and Communication Technology** | |
|---|---|---|
| **Subject: Artificial Intelligence (01CT0616)** | **Aim:** To study N-gram model | |
| **Experiment No: 7** | **Date:** | **Enrolment No: 92301733012** |

```
for w1, w2 in bigrams:
  freq_dict[w1][w2] += 1

# Convert to probabilities
prob_dict = {}

for w1 in freq_dict:
 total_count = sum(freq_dict[w1].values())
prob_dict[w1] = {w2: freq_dict[w1][w2]/total_count for w2 in freq_dict[w1]}

# Function to predict next word
def predict_next_word(word):
 if word in prob_dict:

   return max(prob_dict[word], key=prob_dict[word].get)
 else:
   return "No prediction available"
# Test with seed words
print("Next word after 'i':", predict_next_word("i"))
print("Next word after 'love':", predict_next_word("love"))
print("Next word after 'machine':", predict_next_word("machine"))
```

Results:

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
True
```

```
Next word after 'i': No prediction available
Next word after 'love': No prediction available
Next word after 'machine': No prediction available
```